FORENSIC CODEBASE AUDIT · SOURCE OF TRUTH FOR MANUSCRIPT

SafeHer — Actual System Inventory

A ground-truth inventory of what the SafeHer VAWC survivor-support platform's codebase actually contains and does, as of the current repository state — not what any prior prompt, plan, or design document described. Every claim below is traced to a file path, and every gap is marked rather than guessed.

BASELINE AS OF

2026-09-10 — reconciled to the final verified implementation

DEVELOPMENT HISTORY

2026-09-06 (original audit) → 09-07 (shelter_staff fix, notification/push) → 09-08 (RBAC finalization) → 09-09 (law_enforcement) → 09-10 (this consolidation pass)

REPOSITORY

`safeher-mobile`, branch `main`

METHOD

Direct file inspection, executed `tsc` builds, and live verification against the linked Supabase project (real accounts, real RLS, real Edge Function/pg_net/Expo Push API calls)

SCOPE

`mobile/`, `admin/`, `supabase/`

MODIFICATION

This pass reconciles every section below against the final implementation — it edits the body text directly rather than appending another dated addendum

This document now describes the final, current SafeHer architecture — an 8-role system (§4) including an implemented, live-verified `law_enforcement` role, with `super_admin` retired (kept only as a dormant, non-creatable Postgres enum label for backward compatibility — never an active role). Prior dated findings (shelter_staff fix, RBAC finalization, law-enforcement build) remain as a dated audit trail inside the sections they affect and in the dedicated Development History sections (§31, §32) — they are evidence for how the current state was reached, not a separate or competing description of the system. Where this pass found the earlier text describing an already-superseded state (e.g. an old role count, an unimplemented law-enforcement note, a "10 SQL files" tally), the text was corrected in place. SMS, automated phone calls, external PNP/police-system integration, and universal browser Web Push remain `NOT IMPLEMENTED — OUT OF CURRENT SCOPE`.

---

01 Executive Summary

SafeHer is a two-app system — a React Native/Expo **survivor-facing mobile app** and a React/Vite **administrator web dashboard** — sharing one Supabase Postgres backend (Auth, Realtime, Storage, Edge Functions). The codebase implements a genuinely working, non-trivial system: authentication with email-OTP verification, survivor case reporting with a three-tier administrative escalation hierarchy (Barangay → Municipality → Province), a multi-tenant shelter module with its own admin/staff roles, a referral/assignment-scoped law-enforcement role, a GPS-based SOS trigger with hybrid in-app/push notification fan-out, and a four-layer AI support assistant (rule-based safety triage → FAQ → guided intents → Groq LLM) with human-expert handoff.

SafeHer's final, implemented role architecture has **8 active roles**: `survivor`, `barangay_admin`, `municipal_admin`, `provincial_admin`, `law_enforcement`, `shelter_admin`, `shelter_staff`, `support_expert` (full detail in §4). A ninth Postgres enum label, `super_admin`, still exists in the database for backward compatibility but is **retired and non-creatable** — it is not an active role and should never be described as one.

Five findings matter most for accuracy in the manuscript:

1. **Emergency SOS creates a database record and fans out an in-app notification (plus a best-effort push attempt) to authorized admins**, and — only when an authorized admin explicitly refers it — to a specific assigned `law_enforcement` officer.

It does not send an SMS, place a phone call, integrate with any external police system, or guarantee human response — see §11.

2. **FIXED:** the `shelter_staff` role's account-creation defect (a trigger left over from an earlier module rejected the new role's `shelter_id`) has been corrected and verified live — real account creation, real login, real cross-shelter RLS isolation. See §7's dated finding.
3. **IMPLEMENTED:** a hybrid notification architecture — database-backed in-app notifications remain the authoritative record, and Expo push notifications are dispatched server-side for high-priority event types. The server-side pipeline (trigger → Edge Function → Expo Push API) is verified live end-to-end; **actual on-device delivery to a physical phone remains unverified** because this Expo project has no linked EAS project — an environment/tooling limitation, not a code defect. See §17.
4. **IMPLEMENTED:** `law_enforcement` is a real, built, and live-tested role (21 live test scenarios, zero vulnerabilities) using a narrow, assignment/referral-based access model — no shared queue, no self-assignment, no officer reassignment, no jurisdiction browsing, and no integration of any kind with an external PNP/police database or API. See §32.
5. **SMS, automated phone calls, and external law-enforcement system integration remain entirely absent** — explicitly out of scope, not merely unimplemented by omission (see §17, §24, §32).

The system is best described to a reader as a *case-management and communication platform with a database-backed SOS alerting and hybrid notification mechanism*, not as an emergency-dispatch or rescue system — the codebase does not support the latter claim, and push delivery is a best-effort alert layered on top of an always-authoritative database record, never a substitute for it.

02 Project Structure

AREA	PATH	PURPOSE	STATUS
Mobile app	`mobile/`	Survivor-facing Expo/React Native client — the only client type found; no admin/staff role ever authenticates here	IMPLEMENTED
Admin dashboard	`admin/`	React/Vite web app for all 7 non-survivor roles	IMPLEMENTED
Shared types	`shared/database.types.ts`	Canonical TypeScript schema types, manually re-synced into both apps	IMPLEMENTED
Database (SQL)	`supabase/*.sql` — 12 flat files, no CLI `migrations/` folder	Hand-authored, hand-run schema/RLS scripts (`00_schema` through `module9` — added since the original audit: `module8` notifications/push, `module9` law_enforcement)	IMPLEMENTED

AREA	PATH	PURPOSE	STATUS
Edge Functions	`supabase/functions/`	3 Deno functions: `create-admin-user`, `safeher-chatbot`, `send-push-notification` (added since the original audit). No `config.toml` /import map found.	IMPLEMENTED
Environment config	`admin/.env.example`, `mobile` uses `app.json` extras	Anon key + URL only; `.env` is gitignored/untracked	IMPLEMENTED
Tests	—	No test directories or files found in `mobile/` or `admin/`	NOT FOUND
Documentation	`AGENTS.md`, `CLAUDE.md`, `README.md`	Points to Expo SDK 57 docs; no architecture/requirements doc found	PARTIAL

Inside each app the layering is conventional and consistent: `screens/` or `pages/` (UI) → `contexts/` (auth/theme state) → `lib/` (Supabase client, hooks, helpers) → Supabase (Postgres + RLS). Mobile additionally has `navigation/` (React Navigation stacks/tabs); admin has `routes/` + `layout/` (React Router + shell chrome).

03 Actual Technology Stack

Mobile (`mobile/package.json` , `app.json`)

- Expo `~57.0.20` , React Native `0.86.3` , React `19.2.3` , TypeScript (plain, non solution-style `tsconfig.json`)
- Navigation: React Navigation (stack + custom tab bar, `ClayTabBar.tsx`)
- Backend client: `@supabase/supabase-js@2.45.4`
- Location: `expo-location` ; Biometrics: `expo-local-authentication` ; Files: `expo-image-picker` , `expo-document-picker` ; Maps: `react-native-maps@1.27.2`
- State/forms: local component state and context only — no Redux/Zustand/React Hook Form found
- Android permissions declared: `ACCESS_FINE_LOCATION` , `ACCESS_COARSE_LOCATION` , `CAMERA` , `READ_EXTERNAL_STORAGE` . **No notification permission declared.**

Admin (`admin/package.json`)

- Vite 5 + React 18 + TypeScript 5.5
- Routing: `react-router-dom@6.26.2`
- Charts: `chart.js` / `react-chartjs-2` ; Maps: `@react-google-maps/api` ; Icons: `lucide-react`
- Backend client: `@supabase/supabase-js@2.45.4`

- `admin/tsconfig.json` is **solution-style** (empty `files`, only `references`) — a plain `tsc --noEmit` against it checks zero files; the real check is `tsc -b` against `tsconfig.app.json`. Verified live: `npx tsc -b --noEmit` → 0 errors.

Backend

- Supabase: PostgreSQL, Auth (PKCE flow, mobile-side), Realtime (`postgres_changes` channels), Storage, Edge Functions (Deno)
- No separate application server, message queue, or cache layer found anywhere in the repository

External services actually referenced

SERVICE	PURPOSE	WHERE USED	VERIFIED?
Supabase	DB / Auth / Realtime / Storage / Functions	Both apps, throughout	Yes
Groq (`api.groq.com`)	LLM inference for AI assistant, model `openai/gpt-oss-120b`	`supabase/functions/safeher-chatbot/providers/groq.ts:6,12`	Yes
Google Maps JS API	Shelter map/location picker (admin only)	`admin/src/lib/useGoogleMaps.ts`, `components/maps/*`	Yes
Native device maps (react-native-maps)	Shelter locator + SOS location map (mobile)	`mobile/src/screens/shelters/ShelterLocatorScreen.tsx`	Yes
Expo Push API	Best-effort push alert for high-priority notification events	`supabase/functions/send-push-notification/index.ts`, `mobile/src/lib/pushNotifications.ts`	Server pipeline: Yes. live. On-device delivery: `UNVERIFIED — ENVIRONMENT LIMITATION` (no EAS project linked)
SMS gateway / Twilio-type service	—	—	`NOT IMPLEMENTED — OUT OF SCOPE`
Automated telephony / voice call service	—	—	`NOT IMPLEMENTED — OUT OF SCOPE`
External PNP / police case-management system or API	—	—	`NOT IMPLEMENTED — OUT OF SCOPE` — see §32

04 User Roles

The `user_role` Postgres enum was extended six separate times, each requiring its own standalone `ALTER TYPE … ADD VALUE` execution (`module3/4/6/7/9, Part A`). **Eight roles are active today**: `survivor`, `barangay_admin`, `municipal_admin`, `provincial_admin`, `law_enforcement`, `shelter_admin`, `shelter_staff`, `support_expert`. A ninth enum label, `super_admin`, still physically exists in the database (added at `00_schema.sql`, before this project ever adopted a barangay/law-enforcement hierarchy) but is **retired and non-creatable** as of the 2026-09-08 RBAC finalization pass (§31) — it is listed separately below, not among the active roles, and must never be described as a current SafeHer role.

ROLE	WHERE DEFINED / ASSIGNED	TIER	DASHBOARD	KEY RESTRICTION
`survivor`	Default on signup (`handle_new_user()` trigger)	—	Mobile app only	Never reaches the admin app — force-signed-out if it tries (`admin AuthContext.tsx:113-122`)
`barangay_admin`	`create-admin-user`, by a `municipal_admin` (own municipality) or province tier	`barangay`	MunicipalDashboard (scope="barangay")	Scoped to own `barangay_id` (narrower than `municipal_admin`'s `municipality_id` — a real sub-municipal tier, not a duplicate); cannot create any admin accounts
`municipal_admin`	Created via `create-admin-user` by province tier or another municipal_admin (own municipality)	`municipality`	MunicipalDashboard	Scoped to own `municipality_id` by RLS
`provincial_admin`	`create-admin-user`, by another province-tier account (confirmed live — see §31)	`province`	ProvincialDashboard	Province-wide oversight; the only tier that can create `shelter_admin` / `support_expert` / `law_enforcement`
`law_enforcement`	`create-admin-user`, province tier only — **IMPLEMENTED & VERIFIED LIVE, 2026-09-09**	none (no jurisdiction column — access is per-referral, not geographic)	LawEnforcementDashboard ("Assigned Referrals")	Sees only cases/SOS alerts an authorized case-admin has explicitly referred *and assigned* to that specific officer; cannot self-assign, reassign, browse, or update `case_reports.status` — see §32
`shelter_admin`	`create-admin-user`, province tier only	none (explicit — not a rung on the province ladder)	ShelterAdminDashboard	Scoped to own `shelter_id`; can create `shelter_staff` only
`shelter_staff`	`create-admin-user`, by owning `shelter_admin` or province tier — **FIXED & VERIFIED LIVE, 2026-09-07**	none	ShelterStaffDashboard	Read shelter-wide; write only items with `assigned_staff_id = self`
`support_expert`	`create-admin-user`, province tier only	none	SupportExpertDashboard	Excluded from `is_admin()`; sees only requests assigned to self

Retired role — `super_admin` (not active). Confirmed by repo-wide grep (§31) to be 100% functionally identical to `provincial_admin` in every RLS policy, helper function, route guard, and authorization check — the only distinction ever found was a cosmetic dashboard label. As of the 2026-09-08 finalization pass, `create-admin-user` explicitly rejects any attempt to create a new `super_admin` account (`400`, "retired — use provincial_admin instead"), and the admin UI's role picker no longer offers it. The Postgres enum label itself was deliberately left in place — dropping a live enum value requires recreating the type, a disproportionate risk for a label with zero live accounts and zero functional payoff — but this is a dormant, non-creatable artifact, not an active role. It should never appear in a role count, a permission matrix, or a description of current SafeHer administrators.

Verification method: role list and tier mapping from `admin/src/contexts/AuthContext.tsx`; routing from `admin/src/pages/dashboard/DashboardRouter.tsx`; role-creation matrix from `supabase/functions/create-admin-user/index.ts`; enum history from all 12 SQL files (§15); `law_enforcement` and the `super_admin` retirement independently re-verified live in §31/§32.

05 Mobile Screens

21 screens/modals were located under `mobile/src/screens` and two navigation-adjacent full-screen components. **No screen is role-gated** — `RootNavigator.tsx:58` branches only on session/profile presence, never on `profile.role`; the app architecturally assumes every account is a `survivor`.

SCREEN	PATH (ABRIDGED)	PURPOSE	BACKEND	STATUS
Splash / Onboarding / Welcome	`auth/SplashOnboardingWelcome.tsx`	First-run carousel + 911 shortcut	none	IMPLEMENTED
Login	`auth/LoginScreen.tsx`	Sign in	`auth.signInWithPassword`	IMPLEMENTED
Register	`auth/RegisterScreen.tsx`	3-step signup	`municipalities:select`, `auth.signUp`	IMPLEMENTED
Email Verification	`auth/EmailVerificationScreen.tsx`	8-digit OTP	`auth.verifyOtp`, `auth.resend`	IMPLEMENTED
Forgot / Reset Password	`auth/PasswordRecovery.tsx`	Link-based reset	`auth.resetPasswordForEmail`, `auth.updateUser`	IMPLEMENTED
Home	`home/HomeScreen.tsx`	Dashboard, SOS entry	`survivors`, `case_reports`, `notifications` (select)	IMPLEMENTED
My Cases / Case Detail	`cases/CaseListScreen.tsx`, `CaseDetailScreen.tsx`	Track own reports	`case_reports`, `case_status_history`, `shelter_referrals` (select only)	IMPLEMENTED (read-only)
SOS	`sos/SosScreen.tsx`	View/activate active alert, live map	`sos_alerts` select + realtime	IMPLEMENTED
Report Incident	`incident/ReportIncidentScreen.tsx`	Submit case + evidence	`case_reports` / `case_status_history` insert, `storage:evidence`	IMPLEMENTED
Shelter Locator / Request Shelter	`shelters/*.tsx`	Find shelters; submit a request	`shelters` select, `shelter_requests` insert	IMPLEMENTED
SafeHer Assistant	`assistant/SafeHerAssistantScreen.tsx`	AI chat + expert handoff	`functions.invoke('safeher-chatbot')`, `chat_messages`, `expert_support_requests`	IMPLEMENTED
Notifications	`notifications/NotificationsScreen.tsx`	In-app inbox	`notifications` select + realtime insert	IMPLEMENTED (pull/realtime, not push)
Profile / Change Password	`profile/*.tsx`	Edit profile, avatar, logout	`storage:avatars`, `user_profiles` / `survivors` update	IMPLEMENTED
Settings Sheet	`components/SettingsSheet.tsx`	Appearance, app-lock, legal	AsyncStorage only, no Supabase	IMPLEMENTED

Two screens are referenced by the working tree's `git status` as deleted (`RegistrationSuccessScreen.tsx`, `chatbot/ChatbotScreen.tsx`) and are not registered in any navigator read for this audit — the live flow uses `EmailVerificationScreen` and `SafeHerAssistantScreen` in their place.

06 Functional Modules

MODULE	USERS	CORE TABLES	STATUS
Authentication & session	All	`auth.users`, `user_profiles`	IMPLEMENTED
Case reporting & tracking	Survivor (create/track), all admin tiers (manage)	`case_reports`, `case_status_history`, `case_assignments`, `evidence`	IMPLEMENTED
Case escalation (org-tier referral)	Barangay/Municipal admins	`case_referrals`	IMPLEMENTED (Phase 1: immediate transfer only)
Shelter → case referral	Any admin (create), shelter admin/staff (action)	`shelter_referrals`	IMPLEMENTED

MODULE	USERS	CORE TABLES	STATUS
Case/SOS → law-enforcement referral	Case-owning admin (create/assign), `law_enforcement` (assigned officer only, action)	`law_enforcement_referrals`	**IMPLEMENTED & VERIFIED LIVE — 2026-09-09** — see §32
Direct shelter request	Survivor (create), shelter admin/staff (action)	`shelter_requests`	**IMPLEMENTED**
Multi-shelter staffing	Shelter admin, shelter staff	`user_profiles.shelter_id`, `assigned_staff_id` columns	**FIXED & IMPLEMENTED — 2026-09-07** — see §7
Emergency SOS	Survivor (trigger), all municipal+ admins (monitor + notified), `law_enforcement` (only if explicitly referred)	`sos_alerts`, `notifications`, `push_tokens`, `law_enforcement_referrals`	**PARTIALLY IMPLEMENTED** — DB alert + in-app/push notification fan-out, still no SMS/call dispatch, no external emergency-service integration
AI support assistant	Survivor	`chat_sessions`, `chat_messages`, `faqs`	**IMPLEMENTED**
Human expert handoff	Survivor, support_expert	`expert_support_requests`	**IMPLEMENTED** (manual assignment only)
Barangay/municipality/org management	Municipal+ admins	`barangays`, `municipalities`	**IMPLEMENTED**
User/role management	Any admin tier (scoped)	`user_profiles`	**IMPLEMENTED**
In-app + push notifications	All	`notifications`, `push_tokens`	**IMPLEMENTED — 2026-09-07** in-app authoritative + push for 5 of 6 types (all but `Announcement`); on-device push delivery unverified pending EAS setup
Reports & analytics	Admin tiers	`case_reports`, `shelter_referrals`, `sos_alerts` (client-aggregated)	**IMPLEMENTED**
Audit logging	System (triggers)	`audit_logs`	**PARTIALLY IMPLEMENTED** — not all tables covered

07 Feature-by-Feature Implementation Audit

Two full traceability chains, plus the audit's most important cross-file finding.

Case Creation (Survivor → Database)

`ReportIncidentScreen` submit button → client validation (case type/description/location non-empty) → `case_reports:insert` (`status:'Pending'` , client-generated `case_number`) → `audit_logs:insert` → `case_status_history:insert` (seeds the timeline) → optional `storage.evidence:upload` + `evidence:insert` → success screen shows the case number. RLS: `"Survivors can submit own cases"` INSERT policy requires `survivor_id` to resolve to the caller via `survivors.profile_id = current_profile_id()` . No missing link found in this chain. `mobile/src/screens/incident/ReportIncidentScreen.tsx:44-154`.

Case Status Update (Admin → Database → Survivor)

`CaseDetailPage` "Update status" button (enabled only if `CASE_STATUS_FLOW[current]` is non-empty) → `case_reports:update({status})` → `case_status_history:insert` → `notifications:insert` to the survivor. No mobile-side path can perform this update — confirmed by an exhaustive grep of

every `.update(` call in `mobile/src`, which found none touching `case_reports`. `admin/src/pages/cases/CaseDetailPage.tsx:137-176`.

FIXED & VERIFIED LIVE — 2026-09-07 `shelter_staff` account creation defect

ORIGINAL FINDING	The `shelter_staff` role had a complete dashboard, RLS model, and Edge Function creation path — but the trigger validating `user_profiles.shelter_id` (`validate_user_profile_shelter()`, originally defined once in `module4_shelter_admin.sql:217-237` and never updated for the new role) rejected any non-`shelter_admin` role with a non-null `shelter_id`, blocking every attempt to create a `shelter_staff` profile with a shelter assigned.
ROOT CAUSE, CONFIRMED LIVE	Querying the live database directly (`pg_get_functiondef`) confirmed the deployed function matched the defective module4 logic exactly, and that `'shelter_staff'` was not yet even a valid `user_role` enum value — `module7_shelter_staff.sql` had never been applied to the live database at all, despite being fully written.
FIX	`module7_shelter_staff.sql` (never-deployed, so corrected in place rather than via a separate patch file) now redefines `validate_user_profile_shelter()` to accept `new.role in ('shelter_admin', 'shelter_staff')`, preserving the same municipality-consistency check for both roles and still rejecting every other role — no security weakening, same validation depth, just the missing branch added.
LIVE VERIFICATION	**Before:** a real HTTP call to the deployed `create-admin-user` function (as a real, freshly-authenticated `provincial_admin` test account) reproduced the exact defect: `400 {"error":"shelter_id may only be set for shelter_admin profiles"}`. **After** applying the fix to the live database: the identical call returned `200` with a correctly populated profile (`role: shelter_staff`, `shelter_id` and `municipality_id` both correctly resolved from the target shelter). The new account then **logged in successfully through the real admin web UI** (Playwright) and its **ShelterStaffDashboard rendered** with the assigned task visible. Cross-shelter RLS isolation was verified directly against PostgREST with two shelters and two independent staff/admin sessions: the staff account could read only its own shelter's requests (0 rows for the other shelter, both via list and direct-by-id fetch), could not self-assign an unassigned request (0 rows affected), could not touch the other shelter's request (0 rows affected), and — once properly assigned by its shelter_admin — could update its assigned request but was explicitly rejected (`"Shelter staff cannot reassign a request"`) when attempting to change the assignment itself. A cross-shelter assignment attempt by the shelter_admin (assigning across shelters) was also correctly rejected. All test accounts and data were removed afterward; the live database returned to its exact prior account/role counts.
CONFIDENCE	HIGH — reproduced, fixed, and re-verified against the live production database and the live deployed Edge Function, not a static-code inference.
MANUSCRIPT USAGE	Multi-shelter staffing (creation, login, shelter-wide read, own-assignment write, cross-shelter isolation) can now be described as an implemented, verified capability.

`IMPLEMENTED — 2026-09-07` **Hybrid notification architecture**

WHAT CHANGED	A new table (`push_tokens`), two new/extended triggers (`notify_on_sos_created` , `dispatch_push_for_notification`), and a new Edge Function (`send-push-notification`) were added (`module8_notifications_push.sql`). The database `notifications` row remains the single authoritative record for every event; a subset of notification types additionally attempt a best-effort Expo push alert.
LIVE VERIFICATION	Triggering a real SOS alert (as a real, RLS-governed survivor account) produced a correctly-scoped in-app notification for province-wide and municipality-matched admins, with generic, non-sensitive copy ("SafeHer Emergency Alert" / "An emergency alert requires your attention.") — and confirmed `shelter_admin` / `shelter_staff` accounts received nothing for it, matching the existing SOS-monitoring role scope. A real staff-assignment event produced an Assignment-type notification, and the full dispatch pipeline (Postgres trigger → `pg_net` → Edge Function, authenticated via a generated internal secret never derived from the platform's own service-role key → Expo Push API) was observed firing end to end against the live, deployed infrastructure, including a real Expo API response and automatic deactivation of a token Expo reported as unregistered.
CONFIDENCE	HIGH for the server-side pipeline (verified live, real network calls). `UNVERIFIED — ENVIRONMENT LIMITATION` for actual on-device delivery — see §17.

Emergency SOS (Trigger → Database → Admin visibility)

SOS button → `preflightLocation()` checks services/permission → `acquireSosLocation()` (GPS Highest→High→last-known fallback, 8s/5s timeouts) → `sos_alerts:insert` (`status:'Triggered'` , lat/lng or `null` , `location_source`) → `audit_logs:insert` . Admin side: `SosMonitoringPage` holds a `postgres_changes` realtime subscription on `sos_alerts` , so a new row appears live *only while an admin has that page open*. **Missing link, by design of the current codebase: nothing between the insert and the admin's browser tab** — no server-side push/SMS/webhook fires on insert. `mobile/src/lib/useSosTrigger.ts:57-79`, `admin/src/pages/sos/SosMonitoringPage.tsx:35-38`.

08 Case Management

FUNCTION	IMPLEMENTATION	ROLES	RLS	STATUS
Creation	Survivor-only, mobile `ReportIncidentScreen` ; admin never inserts (verified by exhaustive grep)	survivor	`"Survivors can submit own cases"`	`IMPLEMENTED`
Statuses (actual, from enum)	`Pending → Under Review → {Assigned, Referred} → In Progress → {Referred, Resolved} → Closed` , per `CASE_STATUS_FLOW` — matches DB enum `case_status` exactly, no invented values			
Tracking (view)	Survivor sees own; admin sees per-tier scope (municipality / own	all	`admin_can_access_case_active( ) / owns_case()`	`IMPLEMENTED`

FUNCTION	IMPLEMENTATION	ROLES	RLS	STATUS
	barangay-owned / province-wide)			
Update (status)	`CaseDetailPage.submitStatusChange`, admin-only	municipal/provincial/barangay admin	`admin_can_access_case_active()`	`IMPLEMENTED`
Assignment	`case_assignments` insert; pool scoped to case owner's tier	same as above	`admin_can_access_case_active()`	`IMPLEMENTED`
History	`case_status_history`, rendered as a timeline; **stores status as free `varchar(50)`, not the enum type**	owner/assigned admin	`owns_case()` or `admin_can_access_case()` (broad, keeps read access after transfer)	`PARTIALLY IMPLEMENTED` — DB doesn't enforce valid values in history rows
Ownership / escalation	`case_referrals` insert (`submitEscalate`) → trigger applies `owner_level` / `owner_barangay_id`	current-owning tier only (`can_refer_case()`)	immutable history, no UPDATE/DELETE grant	`IMPLEMENTED` (immediate-transfer only, no accept/reject)
Law-enforcement referral	`law_enforcement_referrals` insert (`submitLawEnforcementReferral`), optionally with an officer pre-assigned — does not change case ownership or `case_reports.status`	current-owning tier only (`admin_can_access_case_active()`) to create/assign; assigned `law_enforcement` officer only to act on it	immutable target (`case_id` / `sos_id` fixed at creation), no DELETE grant	`IMPLEMENTED & VERIFIED LIVE — 2026-09-09` — see §32
Closure	Terminal `Closed` status, same "Update status" modal — no dedicated close flow	admin	as above	`IMPLEMENTED`

09 Referral System

SafeHer contains **three structurally distinct referral mechanisms** that share no code and must not be conflated in the manuscript:

A. Case → Shelter referral (`shelter_referrals`)

- Created only from `CaseDetailPage.submitReferral` (admin-initiated, requires selecting a specific shelter — the dropdown is mandatory, `disabled={!referralShelter}`); sets the case's status to `Referred` and notifies the survivor.
- Actioned from `ShelterReferralPage.tsx` (any admin, status only) and from `ShelterAdminDashboard` / `ShelterStaffDashboard` (shelter-side accept/decline/complete).
- `referral_status` values: `Pending → {Accepted, Declined}`, `Accepted → Completed`. Accepting bumps `shelters.current_occupancy`.
- **Never notifies the survivor when the shelter itself updates status** — the table has no `profile_id` column to notify against (explicit comment in `ShelterAdminDashboard.tsx:315-318`).

B. Organizational-tier escalation (`case_referrals`)

- Barangay → Municipality → Province transfer of case ownership. Insert-only, immutable, always `status:'Completed'` in the current UI (no accept/reject step — `responded_by` / `responded_at` exist but are never populated by anything in the current code).
- The *only* writer of `case_reports.owner_level` / `owner_barangay_id` is the `apply_case_referral()` trigger — never a direct client update (confirmed: those columns are absent from every GRANT in every SQL file).
- Gated by `can_refer_case()` : only the tier that currently owns the case can create the referral.

C. Case/SOS → Law-enforcement referral (`law_enforcement_referrals`)

- `IMPLEMENTED & VERIFIED LIVE — 2026-09-09` — full detail in §32. Created only by the case's currently-owning admin (or, for an SOS alert, an admin already authorized to act on it); grants a specific, explicitly-assigned `law_enforcement` officer visibility into that one case or SOS alert. Does not transfer case ownership and never changes `case_reports.status` — the referring tier keeps full authority over the case; the officer records their own status/action against the referral row only.
- Reuses the existing `referral_status` enum (`Pending → {Accepted, Declined}` , `Accepted → Completed`) rather than a new one. No shared queue: a referral is invisible to every `law_enforcement` account until a specific officer is assigned to it, and the officer can neither self-assign nor reassign it.

Referral types are exactly these three — no other referral entity (e.g., to an external NGO/BPO, or a direct integration with an external law-enforcement/PNP system) exists anywhere in the schema or UI.

10 Shelter Functionality

ASPECT	FINDING
Shelter records	Full CRUD (`ShelterManagementPage.tsx`) — create/edit/soft-deactivate (`is_active` toggle); **no hard delete found anywhere**. Gated to provincial/super/municipal admin — `shelter_admin` is deliberately excluded from this page.
Shelter staff	`FIXED & VERIFIED LIVE, 2026-09-07` — account creation, login, dashboard rendering, shelter-wide read, own-assignment write, and cross-shelter denial all confirmed against the live database and live UI. See §7.
Shelter assignment (staff → item)	`assigned_staff_id` column on both `shelter_requests` / `shelter_referrals` (added by `module7`); set only by the owning `shelter_admin` via `ShelterAdminDashboard.submitAssign` . `VERIFIED LIVE` — assignment, cross-shelter assignment rejection, and self-reassignment rejection all confirmed (§7). Assignment now also fires an Assignment-type notification with push dispatch (§17).
Shelter availability	`capacity` / `current_occupancy` on `shelters` ; bumped automatically when a request/referral is accepted.

ASPECT	FINDING
Shelter requests (direct)	Survivor-initiated, mobile-only insert path. **`shelter_id` is mandatory at submission** — enforced both by the DB (`NOT NULL`) and client validation (`if (!shelterId) {"{"} setError(...) {"}"}`).
Shelter status	`shelter_request_status` : `Submitted → Under Review → {Awaiting Information ⇄ Under Review, Accepted} → Placement Confirmed → Completed` , with `Declined` / `Cancelled` off-ramps. All 8 values are real enum labels, not invented.
Staff/admin access	Admin: shelter-wide read+write within own `shelter_id` . Staff (if it could be created): shelter-wide read, write only where `assigned_staff_id = self` .
"Pending shelter designation" workflow	`NOT IMPLEMENTED` — explicitly confirmed absent. `module7` 's own header comment states the team inspected the schema and mobile flow and found "no undesignated request pool… to route from," and left both `shelter_id` columns `NOT NULL` and untouched.

11 Emergency SOS

"The SOS trigger itself makes exactly one write: the `sos_alerts` insert." — mobile audit, verified by grep across the entire mobile codebase for telephony/SMS/push call sites.

QUESTION	ACTUAL BEHAVIOR
Trigger	Survivor taps SOS → confirms in `SosConfirmSheet` → location acquired → one row inserted into `sos_alerts`
Location source	Live GPS only (`expo-location` , Highest→High accuracy, then last-known-position fallback up to 10 minutes old). Never a saved or manually entered address. Background location is never requested.
Data collected	`latitude` , `longitude` (nullable), best-effort reverse-geocoded `location_address` , `location_accuracy` (meters), `location_timestamp` , `location_source` enum (`CURRENT` / `LAST_KNOWN` / `UNAVAILABLE`)
Communication	Creates one DB record. This now also fans out a database `notifications` row (title "SafeHer Emergency Alert", no survivor name/address/phone) to every authorized admin and attempts a best-effort Expo push alert to any of their registered devices — `VERIFIED LIVE` . **Still no SMS. No phone call placed automatically. No integration with any external emergency-service or police system.** The admin dashboard's existing Realtime subscription remains a second, independent way admins see a new alert (§17).
Recipients	Province-wide for `provincial_admin` ; municipality-matched for `municipal_admin` / `barangay_admin` — mirrors `SosMonitoringPage` 's existing scoping exactly, no new authorization model invented. `shelter_admin` / `shelter_staff` / `support_expert` confirmed (live test) to receive nothing for an SOS, consistent with their existing exclusion from SOS monitoring. A `law_enforcement` officer receives nothing automatically either — only if one of the authorized admins above explicitly creates a `law_enforcement_referrals` row assigning that specific officer (§32); there is no automatic, jurisdiction-based, or blanket law-enforcement visibility into SOS alerts.
Backend	The alert insert itself is still a direct client insert governed by RLS (unchanged, preserved as-is per instruction). The new fan-out/notification/push behavior runs entirely inside a database trigger (`notify_on_sos_created`) plus one Edge Function (`send-push-notification`) — no change to the mobile app's own SOS-trigger code path.
Resolution	Admin manually advances `Triggered → Responding → Resolved` ; each transition writes an `audit_logs` row and a `notifications` row back to the survivor
Failure behavior	Permission/GPS failure → user can still send with `location_source:'UNAVAILABLE'` via an explicit "Send Without Location" button. Insert failure → error banner with a manual retry button; no automatic fallback to a phone call (only suggested in copy text).

> Manuscript caution: SafeHer's SOS feature should not be described as providing emergency dispatch, rescue, or guaranteed response. The verified

> behavior is *alert creation, database-backed in-app notification, and a best-effort push alert to authorized recipients' registered devices* — a communication mechanism, not a guarantee of human action, and not a substitute for calling local emergency services directly.

12 Artificial Intelligence

ITEM	FINDING
AI feature	Conversational support assistant ("SafeHer Assistant"), one screen, one Edge Function
Provider / model	Groq Cloud, OpenAI-compatible endpoint (`api.groq.com/openai/v1/chat/completions`), model string `openai/gpt-oss-120b` (migrated from the retired `llama-3.3-70b-versatile`)
Input	Survivor's free-text message + session ID, sent only after three deterministic layers fail to produce an answer
Processing order	(1) Rule-based regex safety triage (`safety.ts` — Emergency/Urgent/Support keyword tiers, no network call) → (2) FAQ keyword match against the `faqs` table, returned verbatim if it scores above threshold → (3) deterministic guided-intent matching (case-status / shelter / report intents; case-status intent explicitly refuses to fabricate a status) → (4) Groq LLM as last resort, `temperature:0.4` , JSON-mode output
Output	Reply text, sender type, source tag, a risk level, suggested action chips, and a boolean flag for human-support need
User	Survivor (mobile) only — no admin-facing generative AI feature found
API	`supabase.functions.invoke('safeher-chatbot')` , the only `functions.invoke` / `rpc` call anywhere in the mobile codebase
Safety controls	The AI's own risk classification can only escalate, never downgrade, the rule engine's verdict. Emergency/Urgent keyword matches skip the AI entirely and return a fixed safety panel (SOS/Expert/Shelter actions). The AI's raw JSON output is never trusted as-is — malformed or missing fields fall back to safe defaults.
Actual status	`IMPLEMENTED` — reachable, complete, server-side only. No on-device/client-side AI of any kind was found in the mobile dependency tree.

Human handoff: a confirmation dialog → `expert_support_requests:insert` → optimistic `WAITING_FOR_EXPERT` state → a `support_expert` claims it in `SupportExpertDashboard` → subsequent messages are ordinary `chat_messages` rows with `sender_type:'Expert'` , delivered over the same realtime subscription as AI messages. Routing is **manual only** — `SupportQueuePage.tsx` 's own comment states automatic routing is "explicitly deferred."

13 Authentication

FUNCTION	EXACT SUPABASE CALL	NOTES
Registration	`auth.signUp({email, password, options:{data:{...}}})`	`user_profiles` / `survivors` rows created server-side by the `handle_new_user()` trigger, not client code
Email verification	`auth.verifyOtp({email, token, type:'email'})`	8-digit code, not a magic link; resend via `auth.resend({type:'signup'})`
Login	`auth.signInWithPassword`	On `email_not_confirmed` , redirects to verification instead of an inline error

FUNCTION	EXACT SUPABASE CALL	NOTES
Logout	`auth.signOut`	Confirmation dialog first (mobile); also revocable from other sessions in admin Settings (`signOut({"{"}scope:'others'{"}"})`)
Session persistence	`createClient(..., {"{"}auth:{"{"}storage:AsyncStorage, persistSession:true, autoRefreshToken:true, flowType:'pkce'{"}"}{"}"})`	PKCE flow; deep links handled manually (`detectSessionInUrl:false` , no browser URL bar in RN)
Password reset	`auth.resetPasswordForEmail` → deep link → `auth.updateUser({"{"}password{"}"})`	Link-based (unlike signup, which is code-based); routes through a dedicated Recovery navigator so the old password can't remain active mid-flow
Change password (logged in)	Re-auth via `signInWithPassword` , then `auth.updateUser`	—
OAuth / social login	—	`NOT FOUND IN CODEBASE`
Protected routes (admin)	`ProtectedRoute.tsx` , `requireRole` array	Only covers the 4 province-ladder roles by type; shelter/support roles are gated by which dashboard renders, plus RLS — not by a route-level role array

14 Authorization & Access Control

Authorization is enforced at three independent layers, verified separately: (1) UI-level role checks (can be bypassed by a motivated client, cosmetic only), (2) Postgres GRANTs restricting which columns the shared `authenticated` role can write at all, (3) Row-Level Security policies built on a deliberate two-tier helper-function pattern:

- **Broad, read-only helpers** (`admin_can_access_case()` , `admin_can_access_profile()`) — true for a case's current owner *or* any org that appears in its referral history. Used only where historical visibility is wanted (e.g. status-history SELECT).
- **Narrow, write-gate helpers** (`admin_can_access_case_active()`) — true only for the current owner. Used for every UPDATE/INSERT of operational data.
- All helper functions are `SECURITY DEFINER STABLE` and, with a small number of documented exceptions, are single-hop — they query only `user_profiles` — specifically to avoid RLS self-recursion (Postgres error 42P17). Two classes of exception exist, both deliberate: `admin_can_access_case` / `admin_can_access_profile` (multi-hop, carry an explicit "do not call me from certain policies" warning), and a narrower `set row_security = off` pattern (`can_refer_case()` in `module3` ; `is_case_referred_to_officer()` / `is_sos_referred_to_officer()` in `module9`) used specifically where granting a role visibility into a table would otherwise create a genuine two-table policy cycle. Each `row_security = off` function checks exactly one narrow, single-purpose condition — bypassing RLS internally cannot leak anything beyond that one yes/no fact — and this is applied only at these specific, documented points, not as a general RLS bypass anywhere else in the schema.

TABLE	SELECT	INSERT	UPDATE	DELETE
`case_reports`	owner survivor; scoped admin tier; assigned `law_enforcement` officer (referred cases only, via `is_case_referred_to_officer()`)	survivor only	current-owner admin tier only — `law_enforcement` has no UPDATE path on this table at all	none
`case_referrals`	owner/participant + current or past org	current owner tier only	none — immutable	none
`law_enforcement_referrals`	case/SOS owner-tier admin; assigned officer only (own referrals)	current owner-tier admin only	owner-tier admin (any field); assigned officer (`status` / `action_notes` / `action_taken_at` only, trigger-enforced — never `assigned_officer_id` or the target)	none — immutable history
`shelter_requests`	requester; own shelter's admin/staff; scoped municipal/provincial admin	survivor only	requester (cancel only); shelter admin (full); shelter staff (own assignment only)	none — status-based cancellation instead
`chat_sessions`	participant (survivor/assigned expert)	survivor only	none — trigger-only	none
`audit_logs`	none — no SELECT policy exists at all	own actions only	none	none

Notable enforcement gap: `is_admin()` was extended for `barangay_admin` and `shelter_admin`, and (redundantly, as a no-op) re-declared for the `support_expert` era — but **was never extended for `shelter_staff` or `law_enforcement`**. For `shelter_staff` this is an unremarked gap (no generic admin fallback, e.g. visibility into an inactive shelter row). For `law_enforcement` this is a *deliberate* design choice, not a gap — see the "Data minimization" finding in §32 — since a generic admin fallback is exactly the kind of blanket access the referral/assignment model is built to avoid.

15 Database Schema

21 application tables across 12 hand-run SQL files (`00_schema.sql` then `module1` → `module9`, no Supabase-CLI-managed `migrations/` folder was found — these are manually executed in the Supabase SQL editor, in order).

TABLE	PURPOSE	KEY RELATIONSHIPS	RLS
`municipalities`	Geographic/admin scope root	—	enabled, 3 policies
`barangays`	Sub-municipal admin scope	→ municipalities	enabled, 3 policies
`user_profiles`	All accounts, all roles	→ auth.users, municipalities, barangays, shelters	enabled, 4 policies (no INSERT policy — trigger-only)
`survivors`	Survivor-specific PII	→ user_profiles, municipalities	enabled, 4 policies
`shelters`	Shelter facility records	→ municipalities	enabled, 3 policies
`case_reports`	Incident reports / cases	→ survivors, municipalities, barangays	enabled, 4 policies
`case_assignments`	Staff-to-case assignment	→ case_reports, user_profiles	enabled, 2 policies
`case_status_history`	Status change log	→ case_reports, user_profiles	enabled, 2 policies
`case_referrals`	Org-tier ownership transfer	→ case_reports, barangays, user_profiles	enabled, 2 policies (insert-only, immutable)
`shelter_requests`	Direct survivor→shelter requests	→ shelters, user_profiles, case_reports	enabled, 3 policies
`shelter_referrals`	Case→shelter referrals	→ case_reports, shelters, user_profiles	enabled, 3 policies

TABLE	PURPOSE	KEY RELATIONSHIPS	RLS
`evidence`	Uploaded file references	→ case_reports, user_profiles	enabled, 2 policies
`sos_alerts`	Emergency alert records	→ user_profiles	enabled, 4 policies
`notifications`	In-app notification records	→ user_profiles, case_reports, sos_alerts	enabled, 3 policies
`chat_sessions`	AI/human chat session state	→ user_profiles	enabled, 2 policies (no UPDATE at all)
`chat_messages`	Chat transcript	→ chat_sessions, user_profiles	enabled, 2 policies
`expert_support_requests`	Human-handoff queue	→ chat_sessions, user_profiles	enabled, 3 policies
`faqs`	AI-surfaced canned answers	→ user_profiles (created_by)	enabled, 4 policies
`audit_logs`	Change history	→ user_profiles	enabled, 1 policy (insert-only, no SELECT)
`push_tokens`	One row per (profile, device) Expo push token	→ user_profiles, cascade delete	enabled, 4 policies — self-only for all of select/insert/update/delete
`law_enforcement_referrals`	Case/SOS referred to a specific, assigned `law_enforcement` officer	→ case_reports, sos_alerts (exactly one non-null), user_profiles (referrer + officer)	enabled, 3 policies (select/insert/update, no delete — immutable)

`NEW 2026-09-07` — a 20th table exists outside the exposed `public` schema: `private.app_config(key, value)`, storing the push-dispatch Edge Function URL and an internal shared secret. It has no RLS policy and, critically, no GRANT to `anon` / `authenticated` at all — confirmed live via `has_table_privilege()` returning false for both roles — and is never exposed via PostgREST since PostgREST only serves the `public` schema. It is readable only from inside a `SECURITY DEFINER` trigger function. This exists because Supabase's hosted SQL-execution role does not permit `ALTER DATABASE ... SET` for custom parameters (confirmed live — a `42501 permission denied` error), which is the more commonly documented pattern for this kind of internal config; the table-based approach was substituted for exactly that reason.

Notable schema-level findings

- `case_status_history.previous_status` / `new_status` are `varchar(50)`, not the `case_status` enum — history entries aren't validated against real status values at the DB level.
- `case_reports.owner_level` / `owner_barangay_id` and `user_profiles.barangay_id` / `shelter_id` are deliberately excluded from every UPDATE GRANT — writable only via trigger, service role, or manual SQL. This is a consistent, intentional pattern, not an oversight.
- `municipality_id` on `user_profiles` has no enforcing trigger (unlike `barangay_id` / `shelter_id`, which each have one) — nothing in the schema prevents an inconsistent role/municipality combination there.
- Both the admin and mobile `database.types.ts` files independently omit `faqs` and `expert_support_requests` from the typed `Database.Tables` map, despite both tables being actively queried in each app. This does not currently break either build (Supabase-js degrades gracefully) but removes compile-time row-shape checking for those two tables.

16 Edge Functions / Backend

`create-admin-user`

HTTP POST, JWT-authenticated, looks up the caller's own role via their own RLS-scoped client before doing anything privileged. Full role-creation matrix (verified against the source):

CALLER ROLE	CAN CREATE
`shelter_admin`	`shelter_staff` only, and only for their own shelter (request-body `shelterId` is ignored server-side)
`municipal_admin`	`municipal_admin` or `barangay_admin`, only within their own municipality
`provincial_admin`	`municipal_admin`, `barangay_admin`, `shelter_admin`, `shelter_staff`, `support_expert`, `law_enforcement`, and another `provincial_admin` peer (confirmed live — see §31)
`barangay_admin`	none — excluded from the function's base authorization check entirely
Any role	**`super_admin` is explicitly rejected as a target role** (`400`, "retired — use provincial_admin instead") since the 2026-09-08 finalization pass — verified live. Before that fix, the function had no branch either permitting or denying it, so it silently succeeded by default; this was corrected to an explicit denial, not left as a gap.

Writes: `auth.users` (service-role create), `user_profiles` (service-role update), `audit_logs` (manual insert, since this is the one place that still knows the calling admin's identity for a service-role write). No external service calls. Errors return descriptive JSON with appropriate 400/401/403/405/500 codes, never a raw stack trace. As of a later hardening pass, `municipalityId` / `barangayId` / `shelterId` from the request body are normalized (empty string → `null`) before use, after a real bug surfaced in production where the admin UI sent `municipalityId: ""` for the newly-added no-jurisdiction `law_enforcement` role, causing a raw Postgres `invalid input syntax for type uuid` error — fixed at both the UI call site and, more robustly, at this Edge Function boundary so the same class of bug cannot recur for a future no-jurisdiction role.

`safeher-chatbot`

HTTP POST, JWT-authenticated, session-ownership checked (not role-based). Reads go through the *caller's own* RLS-scoped client; only the final AI/System reply insert uses a service-role client, because the `chat_messages` RLS INSERT policy only permits `sender_type IN ('User','Expert')` for any authenticated caller — `'AI'` / `'System'` are architecturally unreachable except via this one server-side path. Every external-dependency failure (Groq API error, missing API key, malformed AI JSON) degrades to a calm, generic in-character fallback response rather than a raw error — confirmed across four distinct failure branches in the source.

`send-push-notification` NEW — 2026-09-07, verified live

Has no end-user caller — deployed with `--no-verify-jwt` and invoked exactly once, internally, by the `trg_dispatch_push_for_notification` database trigger via `pg_net`, immediately after a high-priority `notifications` row is inserted. In place of a Supabase Auth JWT (there is no "calling user" for a background dispatch), every request must present a generated `x-internal-secret` header matching this function's own `PUSH_DISPATCH_SECRET` environment secret — a value generated fresh for this purpose, never the platform's actual service-role key, so a leak of this secret could only ever be used to invoke this one internal endpoint. Recipient selection is never taken from the request body — the only input accepted is a `notification_id`; the function reads that row back out with its own service-role client and derives the recipient from its already-authorized `profile_id`. It looks up that profile's active `push_tokens`, calls the Expo Push API (`https://exp.host/--/api/v2/push/send`), deactivates any token Expo reports as `DeviceNotRegistered`, and records `push_dispatched_at` / `push_delivery_status` back onto the notification row. Verified live: real trigger firings for both an SOS event and a staff-assignment event correctly reached this function, correctly resolved zero registered tokens for the test accounts used (recorded as `no_recipients`), and — when a syntactically valid but non-existent Expo token was registered — made a real call to Expo's push API, received a real error response, and correctly deactivated that token.

No other Edge Function, config file, or shared module exists in the repository.

17 Notification System

`UPDATED & VERIFIED LIVE — 2026-09-07` SafeHer now implements the approved hybrid architecture: the database `notifications` row is always the authoritative, persistent record; a defined subset of high-priority notification types additionally attempt a best-effort Expo push alert. Push delivery failing or being unconfigured never blocks or removes the notification row.

MECHANISM	TRIGGER	DELIVERY	STATUS
In-app notification (mobile)	DB row insert (trigger or manual)	Fetch-on-focus + Realtime `INSERT` subscription, scoped to own `profile_id`	`IMPLEMENTED`
In-app notification (admin)	Same table	**Fetch-once on mount — not realtime** (unchanged by this update), both the Notifications page and the dashboard bell	`PARTIALLY IMPLEMENTED`
Push notification (Expo, mobile/survivor-facing)	`trg_dispatch_push_for_notification` AFTER INSERT on `notifications`, for types `SOS`, `Case Update`, `Referral`, `Assignment`, `Support`	Postgres trigger → `pg_net` → `send-push-notification` Edge Function (internal-secret authenticated) → Expo Push API	`IMPLEMENTED, server pipeline VERIFIED LIVE` · `on-device delivery UNVERIFIED — ENVIRONMENT LIMITATION`
Push notification (admin/staff, web dashboard)	Same trigger/table	N/A — the admin app is a browser dashboard, not an Expo app; no Web Push (VAPID/service worker) implementation exists	`NOT IMPLEMENTED` — see explanation below
SMS	—	—	`NOT IMPLEMENTED — OUT OF CURRENT SCOPE`

MECHANISM	TRIGGER	DELIVERY	STATUS
Automated phone calls	—	—	`NOT IMPLEMENTED — OUT OF CURRENT SCOPE`
Email (transactional, beyond Supabase Auth's own)	—	—	`NOT FOUND IN CODEBASE`

Priority classification (which types get a push attempt)

The dispatch trigger checks `notification_type` against a fixed list — `Announcement` is the only existing type that stays in-app-only, since it is a broadcast with no individual time-sensitivity. Every other existing type corresponds to a concrete, individually-addressed event:

NOTIFICATION_TYPE	NOW TRIGGERS PUSH?	POPULATED BY
`SOS`	Yes	`notify_on_sos_created` (new trigger, admin recipients) + existing `SosMonitoringPage.updateStatus` manual insert (survivor recipient)
`Case Update`	Yes	Existing manual insert, `CaseDetailPage.submitStatusChange` — unchanged
`Referral`	Yes	Existing manual insert, `CaseDetailPage.submitReferral`; plus `notify_referrer_on_law_enforcement_update` (new, module9 — tells the referring admin when the assigned officer records an update)
`Assignment`	Yes	Existing manual insert (`CaseDetailPage.submitAssignment`) and existing triggers (`notify_on_shelter_request_staff_assignment`, `notify_on_shelter_referral_staff_assignment`); plus `notify_on_law_enforcement_assignment` / `notify_on_law_enforcement_referral_created` (new, module9 — tells the officer when they're assigned)
`Support`	Yes	Existing triggers (`sync_chat_session_from_request`, `notify_on_expert_message`) — unchanged
`Announcement`	No — in-app only	No populating code path found in either app (pre-existing gap, unrelated to this update)

Of these, only `Support` and the shelter-staff meaning of `Assignment` were already populated by a database trigger before this update; `Case Update`, the original `SOS` status-update, the case-assignment meaning of `Assignment`, and `Referral` still rely on the same manual application-level inserts documented in the original audit — this update did not change who creates those rows, only whether a push attempt now follows.

Push payload content (privacy)

The Edge Function deliberately does not forward the notification row's raw `message` field to the push payload for non-SOS types, since several of those fields contain admin- or expert-authored free text (case remarks, referral notes, chat replies). Instead it substitutes a fixed, generic body per type (e.g. "Your case status has been updated. Open SafeHer to view details.") and only ever includes safe identifiers (`notification_id`, `notification_type`, `case_id`, `sos_id`) in the payload's `data` field for deep-linking. SOS notifications are the one type forwarded verbatim, because both places that create an SOS

notification already write safe, non-sensitive copy at the source (no survivor name, narrative, address, or phone number).

Why admin/staff push is not implemented

Every admin and staff role (municipal/provincial/barangay/shelter admin, shelter staff, support expert, law enforcement) uses only the React/Vite web dashboard — confirmed throughout this audit, no admin role has ever used the Expo mobile app. Expo push notifications are delivered only to devices running the compiled Expo app; they cannot reach a browser tab. Reaching a web dashboard would require an entirely separate mechanism (Web Push / VAPID keys / a service worker), which does not exist in this codebase and was judged out of scope for an "Expo-compatible push notification architecture" as requested. The `push_tokens` table and dispatch trigger are role-agnostic — if an admin-capable push client is added in a future phase, it would start working by simply registering tokens into the same table, with no schema change. Until then, admin/staff SOS awareness continues to rely on the pre-existing Realtime subscription (§11) and the newly-added in-app notification row, not push.

18 Security Controls

Implemented security controls (verified)

- Row-Level Security enabled on all 21 application tables, enforced via single-hop `SECURITY DEFINER` helper functions deliberately designed to avoid recursion (with a small number of narrow, documented `row_security = off` exceptions where recursion would otherwise be unavoidable — see below).
- Column-scoped GRANTs prevent the shared `authenticated` Postgres role from writing sensitive columns (`role` , `municipality_id` , `shelter_id` , `barangay_id` , ownership fields) even when a table's RLS UPDATE policy would otherwise allow the row to be touched.
- BEFORE-UPDATE validation triggers add per-role, per-column enforcement that Postgres GRANTs cannot express on their own (e.g., a survivor cannot smuggle a `shelter_requests.status` change through a request they're technically allowed to update for cancellation).
- No service-role credential or hardcoded API key found anywhere in either frontend codebase — confirmed by direct grep of `admin/src` and cross-reference to the one legitimate server-side usage in the Edge Function.

- Groq API key read via `Deno.env.get` server-side only; never echoed to any client response, including debug responses.
- AI output is never trusted as authoritative for sensitive claims — the guided-actions layer explicitly refuses to let the AI fabricate a case status.
- `NEW 2026-09-07` `push_tokens` RLS is strictly self-only for every operation (select/insert/update/delete) — verified live that a second, unrelated user's session receives zero rows when querying another profile's tokens. No admin role, including provincial_admin, has any RLS-granted read access to another profile's push tokens.
- `NEW 2026-09-07` The internal push-dispatch secret and Edge Function URL are stored in a table outside the schema PostgREST exposes, with no grant to `anon` / `authenticated` — verified live via `has_table_privilege()` — rather than as a client-reachable table or a hardcoded literal in a committed SQL file.
- `NEW 2026-09-07` The `send-push-notification` Edge Function authenticates its one legitimate caller (a database trigger) via a purpose-generated shared secret distinct from the platform's actual service-role key, limiting the blast radius of that specific secret if it were ever exposed.
- `NEW 2026-09-08` Admin route-level authorization was hardened after a direct-URL gap was found: five routes (`cases` , `cases/:caseId` , `sos` , `shelters` , `reports` , `users`) previously rendered for any authenticated admin regardless of role, relying solely on RLS to withhold data — confirmed live that no actual data ever leaked (RLS returned zero rows in every case), but the page itself was reachable by an unauthorized role via direct URL entry. Each route now carries an explicit `requireRole` guard matching its real RLS-authorized role set; verified live that an unauthorized role is redirected before the page renders.
- `NEW 2026-09-09` `law_enforcement` 's RLS grants are deliberately minimal — read access only to a case/SOS alert explicitly referred and assigned to that specific officer, never to `survivors` , `user_profiles` (beyond self), or `evidence` , and never a write path to `case_reports.status` . A genuine RLS recursion risk in implementing this (§32) was identified and resolved before deployment using the same narrow, single-purpose `set row_security = off` helper-function pattern already established elsewhere in this schema — applied only at the two specific points that needed it, not as a general RLS bypass.
- `NEW 2026-09-09` A production input-validation bug (an empty-string jurisdiction field reaching a uuid database column as `""` instead of `null` , surfaced when creating the first `law_enforcement` account through the admin UI) was fixed at both the specific UI call site and, more robustly, by normalizing all jurisdiction fields at the `create-admin-`

`user` Edge Function boundary — closing this class of bug for any future no-jurisdiction role, not just this one.

Security best practices not verified

- Storage bucket-level access control (RLS on `storage.objects` for the `evidence` and `avatars` buckets) — `UNVERIFIED`. No bucket creation or storage policy statement appears in any of the 10 audited SQL files; if these buckets/policies exist, they were configured outside the inspected codebase (e.g., directly in the Supabase dashboard).
- Rate limiting on Edge Functions or Auth endpoints — `UNVERIFIED`, not addressed in code (would be a Supabase-platform-level setting if present at all).
- Compliance with any named standard (Data Privacy Act, HIPAA-equivalent, etc.) — `NOT FOUND` in code; no such claim should be made without independent legal review.

19 Storage / File Handling

BUCKET	USED FOR	TRIGGER SCREEN	FILE TYPES	PATH SHAPE
`evidence`	Case-report attachments	mobile `ReportIncidentScreen`	Any image MIME type or PDF, client-constrained via the document picker	`{"{"}case id{"}"}/{" {"}timestamp{"}"}_{" {"}filename{"}"}`
`avatars`	Profile pictures	mobile `ProfileScreen`	Images only (image picker, quality 0.7)	`{"{"}profile id{"}"}/{" {"}timestamp{"}"}.jpg`

Both are reference-based: the `evidence` table stores a `storage_path text not null` column, not a blob — confirmed no `bytea` or equivalent column exists anywhere in the schema. Bucket-level RLS/access policy is `UNVERIFIED` (see §18). No other upload surface (documents, additional attachments, shelter photos) was found in either app.

20 Actual User Workflows

Survivor

Register (3-step) → verify email (8-digit code) → log in → Home dashboard → optionally: report an incident (creates a case, status `Pending`) → track it read-only in My Cases → optionally request a shelter directly (must pick one) → optionally trigger SOS (GPS-based alert row) → optionally open the AI Assistant (safety-checked, FAQ/guided/LLM-answered) → optionally request a human expert → receive

in-app notifications as admins act on any of the above, visible only while the app is open or on next launch.

Shelter Admin

Log in → ShelterAdminDashboard (own shelter only) → review incoming shelter requests/referrals → accept/decline/assign to staff → create `shelter_staff` accounts for their own shelter (fixed and verified live, §7) → manage own shelter's profile/capacity via a separate management flow.

21 Actual Administrator Workflows

Municipal/Provincial/Barangay admin: Log in → tier-scoped dashboard → Case List (scoped by RLS to own jurisdiction) → open a case → update status / assign staff / escalate to the next tier / refer to a shelter / **optionally refer to law enforcement**, choosing a specific officer immediately or leaving it unassigned for a later admin to assign → view SOS monitoring (realtime while open), from which the same law-enforcement referral action is available per alert → manage municipalities/barangays/users within scope → view client-computed reports (case/referral/SOS counts, resolution rate) → respond to survivor's uploaded evidence via signed URLs.

Support Expert: Log in → SupportExpertDashboard → see only requests assigned to self → accept → exchange chat messages with the survivor → close the conversation.

Law Enforcement: Log in → LawEnforcementDashboard ("Assigned Referrals") → see only cases/SOS alerts an admin has explicitly assigned to this specific officer, grouped New/Active/Completed → open one to view the underlying case type/description/incident location or SOS location/timestamp → record action notes and advance the referral's own status (Pending → Accepted/Declined → Completed) → the referring admin is notified when this happens. Cannot browse any other case, cannot see who else exists in the system beyond this officer's own assigned referrals, cannot change the case's own status, cannot assign or reassign the referral.

No workflow step was found for automatic case routing by severity, automated shelter matching, automatic/jurisdiction-based law-enforcement routing, or any BPO/protection-order compliance tracking — none of that exists in the current code.

22 Error & Edge Cases

SCENARIO	ACTUAL BEHAVIOR
Login fails (bad credentials)	Inline error from Supabase Auth; unconfirmed-email case redirects to verification instead of a generic error
Registration fails	Client-side field validation before submit; server error surfaced inline
Network/API unavailable (AI)	Calm, in-character fallback message; dev-only diagnostic logging; never a raw error to the survivor
SOS location unavailable	Explicit "Send Without Location" override path; alert still created with `location_source:'UNAVAILABLE'`
SOS insert fails	Error banner + manual retry button; no automatic phone-call fallback
Unauthorized data access	Blocked at the RLS layer (row simply doesn't return / update silently affects 0 rows) — not implemented as a friendly application-level message in most flows
Admin-side render crash	`AdminErrorBoundary` catches render-phase errors in the routed page only (not the sidebar/shell); shows a generic message + Retry/Back-to-Dashboard, dev-only stack trace
Session expiry	Handled via Supabase's `autoRefreshToken` ; no bespoke expiry UI found beyond the standard sign-out redirect

23 Implementation Status Matrix

FEATURE	STATUS	EVIDENCE	MISSING / PROBLEM
Authentication	IMPLEMENTED	`mobile/AuthContext.tsx` , `admin/AuthContext.tsx`	No OAuth
Case creation & tracking	IMPLEMENTED	`ReportIncidentScreen.tsx` , `CaseDetailPage.tsx`	History status column not enum-typed
Case escalation (org-tier)	IMPLEMENTED	`case_referrals` , `CaseDetailPage.submitEscalate`	No accept/reject step yet (by design, Phase 1)
Shelter referral / request	IMPLEMENTED	`shelter_referrals` / `shelter_requests` , both dashboards	`shelter_referrals` can't notify survivors directly
Law-enforcement role & referral	IMPLEMENTED & VERIFIED LIVE — 2026-09-09	`module9_law_enforcement.sql` , `LawEnforcementDashboard.tsx` , 21 live test scenarios + real UI walkthrough (§32)	On-device push delivery unverified (same EAS limitation as all roles); no external PNP/police system integration exists or is claimed
`super_admin`	RETIRED — non-creatable	`create-admin-user` explicit rejection, admin UI role picker omits it (§31)	Enum value intentionally left in Postgres for backward compatibility; not an active role
Shelter staff role	FIXED & IMPLEMENTED — 2026-09-07	`module7_shelter_staff.sql` (fixed), `ShelterStaffDashboard.tsx` , live before/after verification (§7)	None remaining — previously blocked by a `validate_user_profile_shelter()` trigger gap, now fixed and verified live
Emergency SOS	PARTIALLY IMPLEMENTED	`useSosTrigger.ts` , `SosMonitoringPage.tsx` , `notify_on_sos_created` (new)	No SMS/call dispatch; in-app notification + best-effort push now fan out to admins (2026-09-07), verified live
Hybrid notification + push architecture	IMPLEMENTED — 2026-09-07	`module8_notifications_push.sql` , `send-push-notification` , `mobile/src/lib/pushNotifications.ts`	Server pipeline verified live; on-device delivery unverified pending an EAS project link (environment limitation, not a code defect)
AI assistant	IMPLEMENTED	`safeher-chatbot` , `SafeHerAssistantScreen.tsx`	None found
Human expert handoff	IMPLEMENTED	`expert_support_requests` , `SupportExpertDashboard.tsx`	Manual assignment only
Push notifications	IMPLEMENTED — 2026-09-07	`expo-notifications` dependency, `pushNotifications.ts` , `usePushNotifications.ts` , server pipeline verified live	On-device delivery unverified — no EAS project linked yet (environment limitation)
Barangay hierarchy	IMPLEMENTED	`module3_barangay_hierarchy.sql` , `BarangayManagementPage.tsx`	Sub-roles (MSWDO/Barangay staff), BPO workflow explicitly deferred
Reports/analytics	IMPLEMENTED	`ReportsPage.tsx`	Real but client-computed over unbounded row sets; no pagination evidenced
Audit logging	PARTIALLY IMPLEMENTED	`audit_row_change()` trigger, module3	`survivors` , `case_status_history` , `shelters` have no confirmed audit path in SQL alone

FEATURE	STATUS	EVIDENCE	MISSING / PROBLEM
Legal terms / privacy policy	PLANNED	`LegalModal.tsx` placeholder text	Explicit placeholder, not real policy
Formal "provinces" table	PLANNED / DEFERRED	`municipalities.province` is a plain varchar	Never built as its own entity

24 Implemented vs Partially Implemented vs Planned

Verified implemented

Authentication (incl. email-OTP, password reset, session persistence); case reporting and tracking; case status workflow and assignment; org-tier case escalation; shelter-to-case referral; direct shelter requests; multi-tier admin RLS isolation (province/municipality/barangay); **multi-shelter staffing (creation, login, dashboard, assignment, cross-shelter isolation — fixed and live-verified)**; **the law-enforcement role and its referral/assignment workflow (implemented and live-verified across 21 test scenarios plus a real UI walkthrough)**; admin route-level authorization matching each role's real RLS scope; the AI support assistant and its four-layer safety design; human expert handoff (manual); in-app/realtime notifications (mobile); **hybrid push notifications for high-priority events, server pipeline live-verified end-to-end**; file/evidence and avatar upload; biometric app lock; admin error boundary; client-computed reporting/analytics.

Partially implemented

Emergency SOS (creates alerts, now fans out in-app + push notifications to admins as of 2026-09-07, still no SMS/call dispatch); audit logging (covers privilege/ownership-changing tables, not all tables); admin-side notifications (fetch-once, not realtime — unchanged); push notifications (server pipeline verified live; on-device delivery pending an EAS project link).

Unverified

Storage bucket RLS; actual on-device Expo push delivery (no EAS project linked, no dev build, no physical device available in this environment); notification auto-population coverage for every future code path that might create a `notifications` row.

Planned / future, not implemented

Real Terms of Service/Privacy Policy copy; a formal `provinces` table; MSWDO-staff/Barangay-staff sub-roles; BPO/protective-order compliance workflow; accept/reject step for case escalation; automatic/severity-based case or shelter routing; a "pending shelter designation" pool; Web Push for the admin browser dashboard (deliberately out of scope this phase — see §17).

`Not found / explicitly out of scope`

SMS (`NOT IMPLEMENTED – OUT OF CURRENT SCOPE` , by explicit instruction); automated phone calls (`NOT IMPLEMENTED – OUT OF CURRENT SCOPE` , by explicit instruction); any integration with an external PNP/police database, case-management system, or government API (`NOT IMPLEMENTED – OUT OF CURRENT SCOPE` — `law_enforcement` is a SafeHer-internal role only, see §32); universal browser Web Push for admin/staff (`NOT IMPLEMENTED – OUT OF CURRENT SCOPE`); guaranteed emergency response/rescue of any kind; an active `super_admin` role (retired, see §4/§31); OAuth/social login; on-device AI; any test suite; any hard-delete path for shelters or other core records.

25 Verified System Limitations

- **SOS still has no SMS/call channel** — as of 2026-09-07 an alert generates an in-app notification and a best-effort push attempt to admins, but nothing guarantees a human sees or acts on it promptly; the pre-existing Realtime dashboard subscription remains a second, independent path.
- **Push notifications cannot yet reach a real device** — the server-side pipeline is implemented and verified live end-to-end (trigger → Edge Function → Expo Push API), but this Expo project has no linked EAS project (`extra.eas.projectId`), so `getExpoPushTokenAsync()` cannot yet obtain a real token; additionally, Expo Go itself no longer supports remote push on Android from SDK 53 onward, meaning even a linked project would require a development or production build to test on a real device. This is an account/tooling limitation, not a code defect — see §7/§17 evidence for what could be verified without it.
- **Admin/staff (including `law_enforcement`) have no push-capable client at all** — the admin dashboard is a browser app; Expo push cannot reach it, and no Web Push implementation exists. Admin SOS awareness still depends on the dashboard being open (Realtime) or the admin later checking their in-app Notifications page; a `law_enforcement` officer likewise only learns of a new referral by opening the dashboard or via the in-app notification, not a push alert to a device.
- **`law_enforcement` is a SafeHer-internal role only** — there is no integration of any kind with an external PNP database, police case-management system, or government API. Referral to "law enforcement" means making a case/SOS alert visible to a SafeHer user account with that role, nothing more.
- **Case escalation is blind/immediate** — the receiving tier has no opportunity to accept or reject a transferred case in the current UI. The law-enforcement referral shares this immediate-transfer character (no accept/reject gate before the officer can act), though

the officer's own referral *status* (Pending/Accepted/Declined/Completed) is theirs to set once assigned.

- **No "unassigned" shelter request state exists** — every request/referral must name a specific shelter at submission time.
- **Storage-layer access control could not be verified** from the inspected SQL — evidence and avatar files' real-world access boundary is unconfirmed.
- **Case status history is not type-safe at the database level** — it stores free text, not the constrained enum used by the case itself.
- **Reports/analytics run entirely client-side** over full result sets with no evidenced pagination — scalability at high case volumes is unverified.
- **Legal copy is placeholder text**, not a reviewed Terms of Service or Privacy Policy.
- **No automated test suite exists** in either application, so regressions are only caught by manual testing.

26 Manuscript-Ready Factual System Description

SafeHer is a two-application platform intended to support survivors of violence against women and children (VAWC) in reporting incidents, requesting shelter, triggering an emergency alert, and receiving AI- and human-mediated support, while giving a three-tier administrative hierarchy (barangay, municipal, and provincial administrators, plus shelter-specific and support-expert roles) the tools to track, assign, and escalate cases. A distinct `law_enforcement` role represents an authorized law-enforcement user *within SafeHer* — it is not an integration with any external police or government system.

The platform's final role architecture has eight active roles: `survivor`, `barangay_admin`, `municipal_admin`, `provincial_admin`, `law_enforcement`, `shelter_admin`, `shelter_staff`, and `support_expert`. An earlier, functionally-redundant ninth role, `super_admin`, has been retired — its Postgres enum label remains in the database for backward compatibility only, but it is no longer creatable and is not an active administrator type.

The platform consists of a React Native/Expo mobile application used exclusively by survivors, and a React/Vite web dashboard used by all seven administrative and staff roles, both backed by a single Supabase PostgreSQL database with Row-Level Security, Realtime subscriptions, file storage, and three serverless Edge Functions.

Implemented functionality includes: email-verified account registration and authentication; survivor-initiated incident reporting with optional evidence attachment; a seven-state case status workflow with assignment and a formal, trigger-enforced ownership-escalation mechanism between barangay, municipal, and provincial tiers; a shelter module supporting both survivor-initiated shelter requests and admin-initiated case-to-shelter referrals; a referral/assignment-based law-enforcement module through which an authorized case-management admin can make one specific case or SOS alert visible to one specific, explicitly-assigned law-enforcement officer — never a shared queue, jurisdiction-wide access, self-assignment, or reassignment; a GPS-based emergency SOS mechanism that creates a database record and notifies authorized administrators (and, if separately referred, a law-enforcement officer) through both a live dashboard subscription and a best-effort push alert; and a four-layer AI support assistant (deterministic safety triage, FAQ matching, guided-intent responses, and a Groq-hosted large language model as a last resort) with a manual handoff path to a human support expert.

Access control is enforced through PostgreSQL Row-Level Security policies built on single-purpose helper functions, supplemented by column-scoped database grants and validation triggers that restrict which fields each role may modify. A small number of these helper functions deliberately bypass RLS on one specific internal query (`set row_security = off`) where granting a role narrow visibility would otherwise create a self-referential policy cycle between two tables — applied only at those specific, documented points, not as a general RLS bypass.

SafeHer uses database-backed in-app notifications as a persistent, authoritative notification record, and Expo push notifications for selected high-priority events (emergency SOS, case updates, referrals, assignments, and support-conversation activity) on the survivor-facing mobile app. The server-side push-dispatch pipeline (database trigger → Edge Function → Expo Push API) has been implemented and verified end-to-end, including real interaction with Expo's push service and handling of invalid tokens; **actual delivery to a physical device remains unverified**, because this Expo project is not yet linked to an EAS project — an environment/tooling limitation, not a defect in the implemented pipeline. Administrators, staff, and law-enforcement officers all use only a browser-based dashboard and do not have a push-capable client of any kind; their awareness of new events depends on a real-time dashboard subscription and the same in-app notification record available to mobile users. SafeHer does not send SMS, does not place automated phone calls, does not integrate with any external law-enforcement or emergency-service system, and does not guarantee emergency response — these remain explicitly out of scope.

27 SafeHer Feature-to-Research Traceability

The "Problem Addressed" column is intentionally left for the researcher to complete against verified literature/problem evidence — filling it here would violate the audit's own no-assumptions rule. Only the "Actual Feature" and "Evidence Needed" columns are populated from this audit.

ACTUAL FEATURE	PROBLEM ADDRESSED	EVIDENCE NEEDED	RESEARCH OBJECTIVE	EVALUATION AREA
Incident reporting + case tracking	*researcher to supply*	User acceptance / usability data for report submission flow	*researcher to supply*	Functional suitability
Four-tier case escalation (barangay→province)	*researcher to supply*	Evidence that manual, non-digital escalation was slow/error-prone prior to SafeHer	*researcher to supply*	Process efficiency
Shelter request/referral module	*researcher to supply*	Baseline data on shelter-placement turnaround	*researcher to supply*	Service coordination
GPS-based SOS alert (DB + dashboard only)	*researcher to supply*	Must not be evaluated against emergency-dispatch benchmarks it was not built to meet	*researcher to supply*	Reliability, response latency
AI support assistant with safety triage	*researcher to supply*	Evaluation of AI answer accuracy/appropriateness against the FAQ/guided baseline	*researcher to supply*	AI accuracy, safety
Row-Level Security multi-tenant isolation	*researcher to supply*	Penetration/access-control testing results	*researcher to supply*	Security
Referral/assignment-based law-enforcement access model	*researcher to supply*	Evidence that blanket/jurisdiction-wide law-enforcement access to VAWC case data is a real privacy risk this narrower model addresses	*researcher to supply*	Data minimization, access control

28 Chapter 1 Usage Map

CHAPTER 1 SECTION	SAFEHER EVIDENCE TO DRAW FROM THIS DOCUMENT
Rationale	§1 Executive Summary, §25 Limitations — actual system purpose and its honest boundaries
Literature Review	§6 Functional Modules, §12 AI — concepts (case management, RLS multi-tenancy, LLM-assisted triage) that map to real implemented mechanisms
Synthesis	§24 Implemented vs Planned — where the literature's ideal meets what SafeHer actually ships
Conceptual Framework	§7 Feature-by-Feature Audit — real input→process→output chains, not idealized ones
Research Problem	§25 Limitations, the flagged shelter-staff finding in §7/§10 — genuine open problems, not invented ones
Scope	§2 Project Structure, §6 Functional Modules — actual system boundaries
Delimitations	§24 "Not found"/"Planned" lists, §25 Limitations
Significance	§20/§21 Actual Workflows — real stakeholder-facing capability, stated without overclaiming
Definition of Terms	§4 User Roles (the final 8 active roles and the retired `super_admin`), §15 Database Schema — actual role names, table names, and enum values as they exist in code

29 Unverified Items Requiring Human Confirmation

`RESOLVED` — items from the original list have since been resolved by live testing across three follow-up passes: `shelter_staff` creation was confirmed broken, fixed, and re-verified against the real admin UI and live database; `module7_shelter_staff.sql` was confirmed never applied and has now been applied; SOS alerts now fan out an in-app notification (and attempt push) to admins via a new trigger, independent of the dashboard's Realtime subscription; the redundant `super_admin` role was confirmed and retired (§31); and `law_enforcement` was built and verified across 21 live test scenarios plus a real UI walkthrough, including a genuine RLS-recursion risk that was caught and fixed before deployment (§32). Remaining open items:

1. **Link this Expo project to a real EAS project** (`eas init`, or manually setting `extra.eas.projectId` in `app.json`) and build a development or production build (Expo Go does not support remote push on Android from SDK 53+) to verify actual on-device push delivery. This is the single highest-priority remaining item — the server-side pipeline is verified live, but no real device has ever received a SafeHer push notification.
2. **Storage bucket RLS for `evidence` / `avatars`** — inspect the live Supabase Storage policies directly; still not present in any audited SQL file.
3. **Whether the `handle_new_user()` trigger and case-number generation behave as the mobile code assumes** — confirm directly against the live database function definitions if not already covered by §15.

4. **Whether admin/staff should get a push-capable client in a future phase** (Web Push for the browser dashboard) — a scope decision for the researcher/developer, not a defect; the current architecture supports it without further schema change if pursued.

30 Final Anti-Hallucination Audit

CHECK	RESULT
Inspected actual implementation, not prior prompts/plans	Yes — all 10 SQL files, both Edge Functions, and every screen/page file in both apps were read directly; a prior in-session plan for the barangay hierarchy was treated only as historical context, and its claims were independently re-verified against current code before being included
Verified every major feature with file evidence	Yes — see per-section citations throughout
Distinguished UI from actual backend functionality	Yes — e.g. §7's shelter-staff finding exists precisely because UI/RLS presence was checked against trigger reality
Distinguished DB schema from actual application usage	Yes — e.g. §15 flags two tables present in the DB but missing from both apps' TypeScript type declarations
Distinguished implemented from planned	Yes — §24
Verified roles, RLS, case/referral/shelter/SOS/AI workflows	Yes — §4, §8–§12, §14
Verified authentication and authorization separately	Yes — §13 vs §14
Identified limitations without overclaiming capability	Yes — §11, §25, and the explicit manuscript-caution callouts
Flagged cross-file contradictions	Yes — the `validate_user_profile_shelter()` / `module7` conflict, independently re-verified by direct grep/read rather than trusting a single pass
Marked genuine uncertainty rather than guessing	Yes — §29, plus inline `UNVERIFIED` and `NOT FOUND` badges throughout

This inventory is complete and ready for review. No Chapter 1 content has been drafted from it — per instruction, that step waits for explicit confirmation.

Consolidation note (2026-09-10): a dedicated pass re-read every section of this document against the final verified implementation and edited the body text in place wherever it still described a superseded state (the old role list, an inactive-but-still-listed `super_admin`, an unimplemented law-enforcement note, stale file/table counts). Dated findings from each development phase remain visible as an audit trail within the sections they affect and in §31/§32, rather than being deleted — but the sections' own descriptive text now states the current architecture directly, not a historical snapshot.

31 Development History — RBAC Finalization Audit (2026-09-08)

This is a dated development-history record, not a separate description of the current system — the role architecture it establishes is stated as present-tense fact in §4, §14, and §18. Kept here as the evidence trail for how that final state was reached and verified.

A dedicated audit of the role architecture, run against the live database and live application with real accounts (all removed afterward). This section documents what was found, what was fixed, and what was deliberately left alone.

Current Role Inventory

ROLE	DEFINED	PURPOSE	JURISDICTION SCOPE	PROBLEM FOUND
`survivor`	`00_schema.sql`	Request help, report incidents, SOS, track own case	Own records only (`profile_id`)	None
`municipal_admin`	`00_schema.sql`	Local case management within one municipality	`municipality_id`	None
`provincial_admin`	`00_schema.sql`	Province-wide oversight/coordination	Province-wide	None
`super_admin`	`00_schema.sql`	— (see finding below)	Province-wide	**100% redundant with `provincial_admin`** — retired from new-account creation
`barangay_admin`	`module3`	Case handling for one barangay (sub-municipal tier)	`barangay_id` (narrower than municipal_admin)	None — confirmed a real hierarchy, not a duplicate
`shelter_admin`	`module4`	Manage one shelter's operations + staff	`shelter_id`	None (defect from prior phase already fixed)
`shelter_staff`	`module7`	Execute assigned shelter work	`shelter_id` + own assignment	None
`support_expert`	`module6`	Confidential AI-escalation chat support	Own assigned conversations only	None — confirmed zero overlap with case administration
law_enforcement	—	—	—	`NOT IMPLEMENTED as of this 2026-09-08 audit` — see finding below; built and live-verified the next day, now current fact per §4

FIXED — 2026-09-08 **Redundant role: `super_admin`**

FINDING

Grepped every `.sql`, every admin-app file, and the one Edge Function that creates admin accounts for every occurrence of `super_admin`. In every single RLS policy, helper function, route guard, and authorization branch, it appears only ever paired with `provincial_admin` in an identical `role in (...)` check — never once treated differently. The only distinction found anywhere in the codebase was cosmetic: a display label ("Super Administrator" vs "Provincial Administrator") on one dashboard heading.

CORRECTION TO THE ORIGINAL AUDIT

§7 of this document previously stated "no role in the code path is ever permitted to create `provincial_admin` or `super_admin`." That was imprecise. Live-tested directly: a real `provincial_admin` test account successfully created both a `super_admin` peer and a `provincial_admin` peer via `create-admin-user` — the function has no explicit branch for either, so the absence of a denial check meant it silently succeeded rather than silently failed. This isn't a privilege-escalation bug (a province-tier admin minting a peer province-tier admin gains nothing new), but it does mean the two labels were freely interchangeable, reinforcing that they carry no functional distinction.

FIX

`super_admin` was removed from `UserManagementPage.tsx`'s creatable-role list, and `create-admin-user` now explicitly rejects `role: 'super_admin'` with `400 {"error":"'super_admin' is retired — use 'provincial_admin' instead (identical privileges)."}` — verified live. The Postgres enum value and every existing `role in ('provincial_admin','super_admin')` check elsewhere were deliberately left untouched: dropping a live enum label requires recreating the type (touching every column that uses it), a disproportionately risky operation for a role with zero live accounts and zero functional payoff. This is an application-layer retirement, not a schema change — non-destructive and fully reversible.

CONFIDENCE

HIGH — grep-confirmed across the whole repository, then behaviorally confirmed live before and after the fix.

`NOT IMPLEMENTED — AS OF THIS 2026-09-08 AUDIT` **Law Enforcement — proposed but not currently implemented**

Superseded the very next day — see §32. This finding is preserved verbatim as the accurate point-in-time record of what prompted the law-enforcement build; by 2026-09-09 `law_enforcement` was implemented and live-verified. Do not cite this card alone as the current state.

FINDING	Searched the entire repository (case-insensitive) for `law_enforcement`, "law enforcement", `PNP`, "police", and "Philippine National Police". The only substantive hit is a regex inside the AI safety-triage layer that *detects* a survivor's message mentioning calling the police (`supabase/functions/safeher-chatbot/safety.ts:30`) — message content, not a system role. `README.md:59` explicitly documents that the original ERD's role enum "only has 4 values (no separate social-worker/police/lawyer roles)."
CONCLUSION	Law enforcement is not, and has never been, an actor in SafeHer's implemented data model, RLS policies, dashboards, or workflows. Per instruction, no `law_enforcement` role was fabricated.
WHAT WOULD BE NEEDED TO IMPLEMENT IT	(1) A new enum value + its own `ALTER TYPE ... ADD VALUE` migration step; (2) a jurisdiction/assignment model — likely mirroring `shelter_id` / `assigned_staff_id`: a referral/incident record explicitly routed to a law-enforcement recipient, with its own RLS scoping (own-precinct or own-assigned-case only, never blanket case access); (3) explicit, narrow RLS SELECT/UPDATE policies granting only the fields a police action requires — never full survivor case narratives by default; (4) a `create-admin-user` branch defining who may create this role and under what jurisdiction; (5) a dashboard scoped to "authorized law-enforcement action," not general case administration; (6) a notification type/recipient rule extending the existing hybrid architecture. None of this exists today — implementing it is a genuinely new feature, not a fix.
MANUSCRIPT USAGE	State exactly as: "Law Enforcement — Proposed / Not Yet Implemented." Never state that SafeHer supports, integrates with, or notifies PNP/law enforcement.

Final Permission Matrix (as of this 2026-09-08 audit — the Law Enforcement column below predates §32's build the next day)

Classified strictly against verified code/RLS behavior at the time — not the target conceptual model. The two "NOT IMPLEMENTED" law-enforcement cells were resolved to `✓ (assigned referrals only)` the following day; see §32's own permission detail for the current, superseding state.

CAPABILITY	SURVIVOR	MUNICIPAL/BARANGAY ADMIN	PROVINCIAL ADMIN	LAW ENFORCEMENT	SHELTER ADMIN	SHELTER STAFF	SUPPORT EXPERT
Submit own case/report	✓	—	—	—	—	—	—
Submit SOS	✓	—	—	—	—	—	—
View own case/notifications	✓	—	—	—	—	—	—

CAPABILITY	SURVIVOR	MUNICIPAL/BARANGAY ADMIN	PROVINCIAL ADMIN	LAW ENFORCEMENT	SHELTER ADMIN	SHELTER STAFF	SUPPORT EXPERT
Manage local cases (own jurisdiction)	—	✓ (municipality or barangay scope)	—	—	—	—	—
Province-wide oversight	—	—	✓	—	—	—	—
Receive law-enforcement referral	—	—	—	NOT IMPLEMENTED	—	—	—
Record law-enforcement action	—	—	—	NOT IMPLEMENTED	—	—	—
Manage shelter (own shelter)	—	—	—	—	✓	—	—
Assign shelter staff	—	—	—	—	✓	—	—
Execute assigned shelter work	—	—	—	—	—	✓	—
Specialized support (assigned conversations)	—	—	—	—	—	—	✓
Create peer/subordinate admin accounts	—	✓ municipal admin/barangay_admin, own municipality only	✓ any admin role except itself-as-super admin (retired)	—	✓ shelter staff, own shelter only	—	—

Live Role-Escalation Test Results (15/15 passed, zero vulnerabilities)

#	ATTACK ATTEMPTED	RESULT
1	municipal_admin B reads a case in municipal_admin A's municipality	0 rows — denied
2	municipal_admin A reads a case in its own municipality	1 row — correctly allowed
3	municipal_admin B updates a cross-municipality case	0 rows affected — denied
4	Survivor B reads Survivor A's case	0 rows — denied
5	Survivor attempts self-role-elevation (`role` column)	`403` , blocked at the GRANT level (not just RLS)
6	Survivor attempts to change own `municipality_id`	`403` , GRANT-level block
7	municipal_admin attempts self-elevation to provincial_admin	`403` , GRANT-level block
8	municipal_admin B deactivates municipal_admin A (cross-municipality)	0 rows — denied
9	support_expert reads `case_reports` (any)	0 rows — denied
10	support_expert reads `sos_alerts` (any)	0 rows — denied
11	Survivor A submits a case forging Survivor B's identity	`403` , RLS INSERT policy rejected
12	municipal_admin B reads Survivor A's `notifications` directly	0 rows — denied (no admin-read branch exists at all)
13	municipal_admin B reads Survivor A's `push_tokens`	0 rows — denied
14	Any client reads `private.app_config` via PostgREST	`404` — schema not exposed
15	Direct URL navigation to `/admin/cases` , `/sos` , `/reports` , `/users` , `/shelters` as `shelter_admin`	Redirected to `/admin/dashboard` for all five (fixed this pass — previously ungated at the route level, though RLS already prevented any data leak)

Frontend route-hardening (fixed this pass)

Five admin routes (`cases` , `cases/:caseId` , `sos` , `shelters` , `reports` , `users`) had no `requireRole` guard, even though their nav links were already correctly hidden per role. Direct URL entry could render the page for an unauthorized role — RLS always returned zero rows (confirmed, no actual data leak), but the page chrome itself was reachable. `admin/src/App.tsx` now gates each route to exactly the role set already implied by its RLS scope and existing nav visibility (province-ladder roles for cases/sos/shelters; province-ladder minus barangay for reports/users, matching `user_profiles` RLS exactly). Verified live via Playwright: `shelter_admin` now bounces to `/admin/dashboard` from all five; `provincial_admin` retains full access to all five.

Dashboard Redundancy Audit

Every admin-tier role has a functionally distinct dashboard already (confirmed in §6/§21 of the original audit, re-checked here): Municipal/Barangay Admin manage a case queue scoped to their jurisdiction; Provincial Admin sees province-wide aggregates and municipality/barangay/user management, not a bigger case list; Shelter Admin manages shelter operations + staff; Shelter Staff sees a personal task queue only; Support Expert sees only assigned conversations. No two dashboards were found showing the same data with only a title changed — no redesign was needed here.

Notification/Jurisdiction Preservation

The hybrid notification architecture from the prior phase was not modified. Confirmed live (tests 12–13 above) that notification and push-token recipients remain strictly self-scoped — no admin role, including provincial_admin, has any RLS-granted path to another profile's notifications or device tokens.

Build & Regression

`npx tsc -b --force` (admin, the real solution-style check) — clean, zero errors, both before implementing fixes and after. No mobile files were touched in this pass.

32 Development History — Law Enforcement Build (2026-09-09)

This is a dated development-history record, not a separate description of the current system — `law_enforcement` is stated as an active, implemented role throughout §1, §4, §6, §8, §9, §11, §14, §18, and §21. Kept here as the design rationale and live-verification evidence for that role.

`law_enforcement` is a real, implemented role — previously reported (§31) as "proposed but not implemented," now built, deployed, and verified live against the production database and the real admin UI. `super_admin` remains retired and was re-confirmed non-creatable in this pass.

Design decision — referral/assignment-scoped, not jurisdictional

Per explicit design direction, `law_enforcement` was built with **no** municipality/province/"unit" jurisdiction column — a fresh repo-wide search confirmed no existing law-enforcement organizational concept (precinct, unit, station) anywhere in the codebase, and inventing one would have been fabrication. Access is granted per-referral: a specific case or SOS alert becomes visible to a *specific* officer only once an authorized case-management admin (whoever currently owns the case) creates a `law_enforcement_referrals` row and sets `assigned_officer_id`. There is no shared queue — unlike `shelter_staff` (which gets shelter-wide read visibility because a shelter is a real jurisdiction unit), `law_enforcement` has no such unit to scope a wide-read boundary by, so visibility is strictly "assigned to me" only.

Why a new table, not a reused one

`shelter_referrals` was considered and rejected — its `shelter_id` is `NOT NULL` and its application logic bumps shelter occupancy on acceptance, neither of which applies to a police referral. `case_referrals` was also rejected — it is an ownership-*transfer* mechanism between Barangay/Municipality/Province tiers; referring a case to law enforcement grants a parallel, narrower view without transferring case ownership, and `case_reports.status` remains solely under the referring tier's control. A new, minimal table — `law_enforcement_referrals` — was built instead, reusing the existing `referral_status` enum rather than inventing a new one.

RLS recursion risk, identified and avoided before deployment

RISK — Granting the assigned officer visibility into `case_reports` required a new SELECT policy that checks `law_enforcement_referrals`. But `law_enforcement_referrals`' own policies call `admin_can_access_case_active()`, which queries `case_reports` — a naive implementation would create a genuine policy cycle (case_reports → law_enforcement_referrals → case_reports), which Postgres rejects with a 42P17 "infinite recursion detected in policy" error.

FIX — Two narrow helper functions (`is_case_referred_to_officer`, `is_sos_referred_to_officer`) were declared with `set row_security = off` — the same technique `module3_barangay_hierarchy.sql`'s `can_refer_case()` already uses for an identical class of problem. Each function checks exactly one narrow condition (does a referral exist assigning *this* officer to *this* case/SOS), so bypassing RLS internally cannot leak anything beyond that yes/no fact.

VERIFICATION — Applied live with zero errors, then confirmed directly: an officer with a live assignment successfully read the case row (no recursion error); an officer without one got zero rows. See the test matrix below.

Data minimization

`law_enforcement` was deliberately **not** granted any access to `survivors` or `user_profiles` beyond its own profile, and no access to `evidence` — the referring case-admin remains the point of contact for identity/contact details. It was not added to `is_admin()`, mirroring the `support_expert` precedent (an operational, narrowly-scoped role, not a general administrator). It can never update `case_reports.status` directly — only its own referral row's status/action fields, exactly like every other referral actor in this schema.

Account creation rule

Province-tier-only — the caller must be `provincial_admin` (the `super_admin` label is still checked alongside it in this and every other caller-authorization branch purely for backward compatibility with any pre-existing legacy account; it is not an active, creatable role — see §4/§31), matching the `shelter_admin` / `support_expert` precedent. Implemented as a server-side check in `create-admin-user`, verified live: a `law_enforcement` officer attempting to create another admin account of any kind was rejected with `403`.

Live test matrix (21/21 passed, zero vulnerabilities, zero recursion errors)

#	SCENARIO	RESULT
1	Officer created via real Edge Function call	200, correct role, no jurisdiction columns set

#	SCENARIO	RESULT
2	Officer attempts to create another admin account	403 — denied
3	Officer reads an unreferred case	0 rows — denied
4	Provincial admin creates an unassigned referral	201 — succeeded, no recursion error
5	Officer reads the case while referral is unassigned	0 rows — denied
6	Officer attempts to self-assign the unassigned referral	0 rows affected — denied
7	Provincial admin assigns the officer	200 — succeeded, Assignment notification fired
8	Officer reads the case after assignment	1 row — correctly allowed (recursion-safe function confirmed working)
9	A second, unassigned officer reads the same case / referral row	0 rows both — denied
10	Assigned officer attempts to reassign to the other officer	Trigger rejection: "Law enforcement cannot reassign this referral"
11	Assigned officer updates status + action_notes (permitted fields)	200 — succeeded
12	Assigned officer attempts to update `case_reports.status` directly	0 rows affected — denied
13	Officer attempts self-elevation to `provincial_admin`	403, GRANT-level block
14	Officer reads unrelated `sos_alerts`	0 rows — denied
15	Officer reads another officer's notifications	0 rows — denied
16	Officer's own Assignment notification, incl. push dispatch bookkeeping	Present, `push_delivery_status: 'no_recipients'` (no device registered — expected)
17	SOS-path: officer reads an unreferred SOS alert	0 rows — denied
18	SOS-path: provincial admin creates a referral with `sos_id` set and an officer pre-assigned	201 — succeeded
19	SOS-path: officer reads the SOS alert after assignment	1 row, full alert detail visible — correctly allowed
20	Regression: `super_admin` still non-creatable	400, same retirement message as §31
21	Regression: unrelated role (`shelter_admin`) creation unaffected	200 — succeeded, unchanged

Live UI test (Playwright, zero page errors)

Real browser flow, not just API calls: a provincial admin opened a real case, clicked "Refer to law enforcement," selected a real officer from a live-populated dropdown, and submitted — the case's "Law-enforcement referrals" card immediately showed the officer's name. The officer then logged in, landed on a dashboard showing "Assigned Referrals" with the real case number, opened it to see the actual case type/description, and recorded an action (status → Accepted + notes) successfully. Zero console/page errors on either side, confirming the PostgREST foreign-key disambiguation syntax (`user_profiles!law_enforcement_referrals_assigned_officer_id_fkey` , required because the table has two FKs to `user_profiles`) resolves correctly at runtime, not just at compile time.

Files changed

`supabase/module9_law_enforcement.sql` (new), `supabase/functions/create-admin-user/index.ts` , `shared/database.types.ts` + its two synced copies, `admin/src/pages/dashboard/LawEnforcementDashboard.tsx` (new),

`admin/src/pages/dashboard/DashboardRouter.tsx`, `admin/src/contexts/AuthContext.tsx`, `admin/src/layout/AppLayout.tsx`, `admin/src/pages/cases/CaseDetailPage.tsx`, `admin/src/pages/sos/SosMonitoringPage.tsx`, `admin/src/pages/users/UserManagementPage.tsx`.

Build & regression

`npx tsc -b --force` clean before and after. No changes to mobile. Notification architecture (§17), push dispatch (module8), and every previously-verified `shelter_staff` / `shelter_admin` boundary (§7/§31) were re-confirmed intact and untouched by this pass — the generic `dispatch_push_for_notification` trigger required zero changes to cover the new role's notifications, exactly as the "reuse existing architecture" design goal intended.

Remaining/unverified

- On-device push delivery to a real `law_enforcement` device remains subject to the same environment limitation already documented in §17 (no EAS project linked) — unchanged by this pass.
- The `notify_referrer_on_law_enforcement_update` coordination notification (tells the referring admin when the officer updates status) was implemented per the trigger logic but not independently re-verified in the live test pass above — it shares the exact same trigger/notification mechanism already proven working for the assignment notification (test 7/16), so confidence is HIGH but this specific path was not separately clicked through.
- All test data (6 accounts, 2 cases, 1 SOS alert, referral rows) was created and fully removed; live role counts were confirmed to match the exact pre-test baseline after every round.